



Universidade de Aveiro

Departamento de Eletrónica, Telecomunicações e Informática

Linguagens Formais e Autómatos / Compiladores

Exercícios-modelo, teste teórico

(Ano Letivo de 2025-2026)

NºMec:

Nome:

Curso:

ALGORITMO do predict:

$$\text{predict}(A \rightarrow \alpha) = \begin{cases} \text{first}(\alpha) & \text{se } \varepsilon \notin \text{first}(\alpha) \\ (\text{first}(\alpha) - \{\varepsilon\}) \cup \text{follow}(A) & \text{se } \varepsilon \in \text{first}(\alpha) \end{cases}$$

ALGORITMO do first:

```

first( $\alpha$ ) {
  if ( $\alpha = \varepsilon$ ) then
    return  $\{\varepsilon\}$ 
   $h = \text{head}(\alpha)$     # com  $|h| = 1$ 
   $\omega = \text{tail}(\alpha)$    # tal que  $\alpha = h\omega$ 
  if ( $h \in T$ ) then
    return  $\{h\}$ 
  else
    return  $\bigcup_{(h \rightarrow \beta_i) \in P} \text{first}(\beta_i \omega)$ 
}

```

ALGORITMO do follow:

1. $\$ \in \text{follow}(S)$
2. if $(A \rightarrow \alpha B \in P)$ then
 $\text{follow}(B) \supseteq \text{follow}(A)$
3. if $(A \rightarrow \alpha B \beta \in P) \wedge (\varepsilon \notin \text{first}(\beta))$ then
 $\text{follow}(B) \supseteq \text{first}(\beta)$
4. if $(A \rightarrow \alpha B \beta \in P) \wedge (\varepsilon \in \text{first}(\beta))$ then
 $\text{follow}(B) \supseteq ((\text{first}(\beta) - \{\varepsilon\}) \cup \text{follow}(A))$

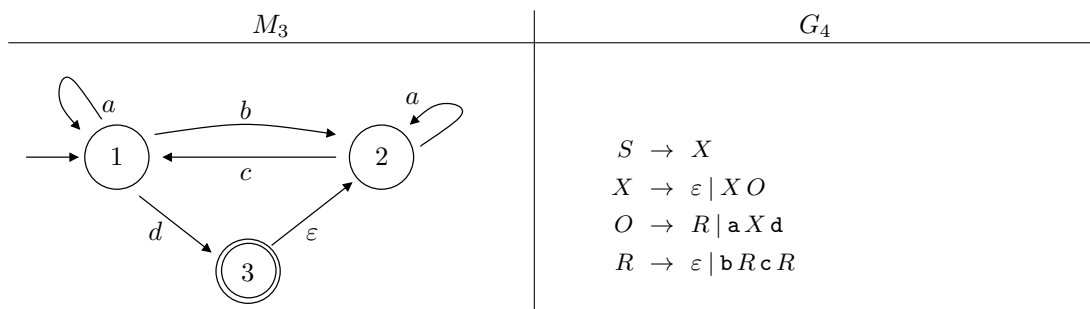
1. Considere, sobre o alfabeto $T = \{a, b, c, d\}$, as linguagens L_1 , L_2 , L_3 , e L_4 definidas da seguinte forma:

$$L_1 = \{a^n(bc)^k d^n : n \geq 0 \wedge k > 0\}$$

$$L_2 = \{w \in T^* : w \text{ é gerada pela expressão regular } e_2 = a^*(b|c)^*(cd)^*\}$$

$$L_3 = \{w \in T^* : w \text{ é reconhecida pelo autómato } M_3\}$$

$$L_4 = \{w \in T^* : w \text{ é gerada pela gramática } G_4\}$$



- (a) Mostre que $abcd \in L_3 \cap L_4$.^{1,2}
- (b) Mostre que $L_1 \subset L_4$.
- (c) Determine uma gramática regular que represente a linguagem L_2 .
- (d) Determine uma expressão regular que represente a linguagem $L_3.L_2$ (concatenação de L_3 com L_2). Apresente o raciocínio e/ou os passos intermédios usados para chegar à sua resposta.
- (e) Construa um autómato finito que reconheça a linguagem $\overline{L_3}$ (complementar de L_3). Apresente o raciocínio e/ou os passos intermédios usados para chegar à sua resposta.
- (f) Projecte uma gramática independente do contexto que represente a linguagem L_1 .

¹Mostra-se que uma palavra ω pertence à linguagem reconhecida por um autómato M , apresentando-se um caminho sobre M que através de ω vá do seu estado inicial a um estado de aceitação.

²Mostra-se que uma palavra ω pertence à linguagem descrita por uma gramática G , apresentando-se uma derivação ou árvore de derivação que transforme o seu símbolo inicial em ω .

- (g) Determine o conjunto $\text{follow}(O)$. Apresente os passos intermédios e/ou o raciocínio adequados para suportar a sua resposta. A simples apresentação da resposta final não será cotada.
- (h) A gramática G4 não é adequada à implementação de um reconhecedor descendente com *lookahead* de 1. Diga porquê e altere-a de forma a obter uma equivalente que o permita. Basta transcrever as partes alteradas.

2. Sobre o alfabeto $T = \{c, t, s, i, =, e\}$ considere a gramática G dada a seguir, cujos conjuntos follow dos seus símbolos não terminais também são dados.

gramática		follow
S	$\rightarrow D$	$\text{follow}(S) = \{\$ \}$
D	$\rightarrow t L_1$	$\text{follow}(D) = \{\$ \}$
	$\mid c t L_2$	
L_1	$\rightarrow V_1$	$\text{follow}(L_1) = \{s, \$ \}$
	$\mid L_1 s V_1$	
L_2	$\rightarrow V_2$	$\text{follow}(L_2) = \{s, \$ \}$
	$\mid L_2 s V_2$	
V_1	$\rightarrow i$	$\text{follow}(V_1) = \{s, \$ \}$
	$\mid V_2$	
V_2	$\rightarrow i = e$	$\text{follow}(V_2) = \{s, \$ \}$

Considere ainda o conjunto de estados (conjuntos de itens) usado na construção de um reconhecedor (*parser*) ascendente, parcialmente apresentada a seguir, e onde $\delta(Z_i, a)$ representa a função de transição de estado.

$$Z_0 = \{ S \rightarrow \bullet D \$, D \rightarrow \bullet t L_1, D \rightarrow \bullet c t L_2 \}$$

$$Z_1 = \delta(Z_0, D) = \{ S \rightarrow D \bullet \$ \}$$

$$Z_2 = \delta(Z_0, t) = \{ D \rightarrow t \bullet L_1, L_1 \rightarrow \bullet V_1, L_1 \rightarrow \bullet L_1 s V_1, V_1 \rightarrow \bullet i, V_1 \rightarrow \bullet V_2, V_2 \rightarrow \bullet i = e \}$$

$$Z_3 = \delta(Z_0, c) = \{ D \rightarrow c \bullet t L_2 \}$$

$$Z_4 = \delta(Z_2, L_1) = \{ D \rightarrow t L_1 \bullet, L_1 \rightarrow L_1 \bullet s V_1 \}$$

$$Z_5 = \delta(Z_2, V_1) = \{ L_1 \rightarrow V_1 \bullet \}$$

$$Z_6 = \delta(Z_2, i) = \{ \dots \}$$

$$Z_7 = \delta(Z_2, V_2) = \{ \dots \}$$

$$Z_8 = \delta(Z_3, t) = \{ \dots \}$$

$$Z_9 = \delta(Z_4, s) = \{ \dots \}$$

...

- (a) Trace as árvores de derivação das palavras “c t i = e” e “t i s i = e”.
- (b) Preencha as linhas da tabela de reconhecimento (*parsing*) para um reconhecedor ascendente relativamente aos estados Z_0 a Z_5 .

	c	t	s	i	=	e	\$		D	L_1	L_2	V_1	V_2
Z_0													
Z_1													
Z_2													
Z_3													
Z_4													
Z_5													

- (c) Determine o valor dos estados (conjuntos) Z_6 a Z_9 e de mais 2 à sua escolha.
- (d) A gramática G representa uma abstração de uma declaração de constantes e variáveis. Sabe-se que:

- o símbolo terminal **c** indica que se trata de uma declaração de constantes;
- o símbolo terminal **t** representa o tipo e possui um atributo chamado **type** que representa o tipo específico que lhe está associado.
- o símbolo terminal **i** representa o identificador de uma variável ou constante e tem um atributo chamado **name** que representa o nome que lhe está associado.
- o símbolo terminal **e** representa uma inicialização e tem um atributo chamado **value** que representa o valor da constante ou valor inicial da variável a que está associado.
- se dispõe de uma função de manipulação de uma tabela de símbolos para inserções de novas entradas (variáveis ou constantes), com a assinatura **addsym(cv, nm, tp, vl)**, onde
 - **cv** é um valor booleano, que indica se se trata da inserção de uma contante ou de uma variável.
 - **nm** representa o nome da variável;
 - **tp** representa o tipo específico;
 - **vl** representa o valor a atribuir à variável/constante, devendo ser omitido no caso de não haver inicialização.

Complete a tabela seguinte de modo a implementar uma gramática de atributos que permita invocar a função **addsym** nos sítios indicados e de forma adequada por cada constante ou variável declarada. Acrescente os atributos dos símbolos não terminais que considere pertinentes.

Produção	Regras semânticas
$S \rightarrow D$	
$D \rightarrow \mathbf{t} L_1$	
$D \rightarrow \mathbf{c} \mathbf{t} L_2$	
$L_1 \rightarrow V_1$	
$L_1 \rightarrow L_1 \mathbf{s} V_1$	
$L_2 \rightarrow V_2$	
$L_2 \rightarrow L_2 \mathbf{s} V_2$	
$V_1 \rightarrow \mathbf{i}$	addsym(
$V_1 \rightarrow V_2$	
$V_2 \rightarrow \mathbf{i} = \mathbf{e}$	addsym(